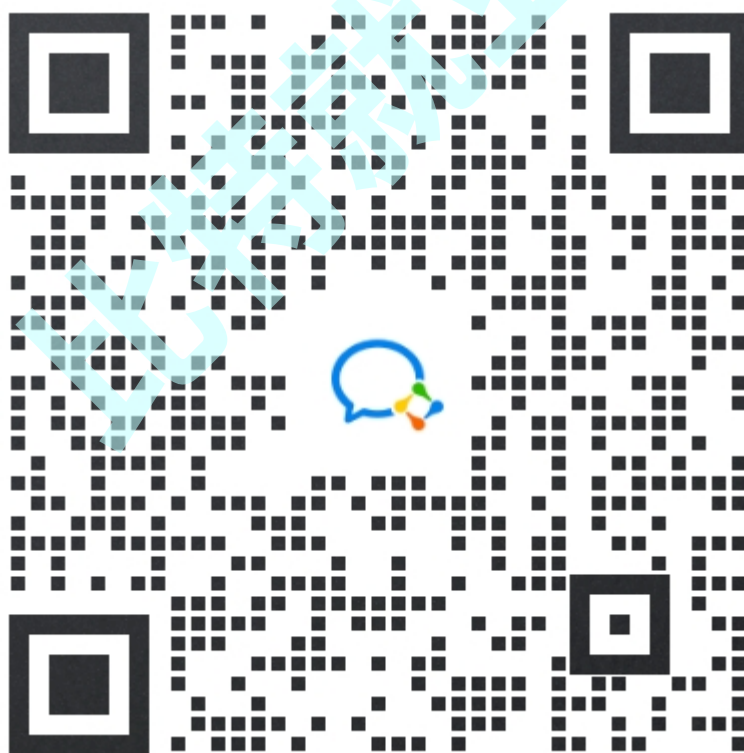


C++ - 高性能内存池

版权说明

本“比特就业课”项目（以下简称“本项目”）的所有内容，包括但不限于文字、图片、音频、视频、软件、程序、数据库、设计、布局、界面等，均由本项目的开发者或授权方拥有版权。我们鼓励个人学习者使用本项目进行学习和研究。在遵守相关法律法规的前提下，个人学习者可以下载、浏览、学习本项目的内容，并为了个人学习、研究或教学目的而使用其中的材料。但请注意，未经我们明确授权，个人学习者不得将本项目的内容用于任何商业目的，包括但不限于销售、转让、许可或以其他方式从中获利。此外，个人学习者也不得擅自修改、复制、传播、展示、表演或制作本项目内容的衍生作品。任何未经授权的使用均属侵权行为，我们将依法追究法律责任。如果您希望以其他方式使用本项目的内容，包括但不限于引用、转载、摘录、改编等，请事先与我们联系，获取书面授权。感谢您对“比特就业课”项目的关注与支持，我们将持续努力，为您提供更好的学习体验。特此说明。比特就业课版权所有方

对比特项目感兴趣，可以联系这个微信。



代码 & 板书链接

<https://gitee.com/HGtz2222/bitproject/tree/master/%E9%AB%98%E5%B9%B6%E5%8F%91%E5%86%85%E5%AD%98%E6%B1%A0>

项目介绍

1.这个项目做的是什?

当前项目是实现一个高并发的内存池，他的原型是google的一个开源项目tcmalloc，tcmalloc全称Thread-Caching Malloc，即线程缓存的malloc，实现了高效的多线程内存管理，用于替代系统的内存分配相关的函数（malloc、free）。

我们这个项目是把tcmalloc最核心的框架简化后拿出来，模拟实现出一个自己的高并发内存池，目的就是学习tcmalloc的精华，这种方式有点类似我们之前学习STL容器的方式。但是相比STL容器部分，tcmalloc的代码量和复杂度上升了很多，大家要有心理准备。当前另一方面，难度的上升，我们的收获和成长也是在这个过程中同步上升。

另一方面tcmalloc是全球大厂google开源的，可以认为当时顶尖的C++高手写出来的，他的知名度也是非常高的，不少公司都在用它，Go语言直接用它做了自己内存分配器。所以很多程序员是熟悉这个项目的，那么有好处，也有坏处。好处就是把这个项目理解扎实了，会很受面试官的认可。坏处就是面试官可能也比较熟悉项目，对项目会问得比较深，比较细。如果你对项目掌握得不扎实，那么就容易碰钉子。

[tcmalloc源代码](#)

2.这个项目的要求的知识储备和难度?

这个项目会用到C/C++、数据结构（链表、哈希桶）、操作系统内存管理、单例模式、多线程、互斥锁等方面的知识。难度的话，如果比特难度满级的项目是5星的话，这个项目应该是4星。所以这里应该加一句，勇敢比特，不怕困难!!!

什么是内存池

1.池化技术

所谓“池化技术”，就是程序先向系统申请过量的资源，然后自己管理，以备不时之需。之所以要申请过量的资源，是因为每次申请该资源都有较大的开销，不如提前申请好了，这样使用时就会变得非常快捷，大大提高程序运行效率。

在计算机中，有很多使用“池”这种技术的地方，除了内存池，还有连接池、线程池、对象池等。以服务器上的线程池为例，它的主要思想是：先启动若干数量的线程，让它们处于睡眠状态，当接收到客户端的请求时，唤醒池中某个睡眠的线程，让它来处理客户端的请求，当处理完这个请求，线程又进入睡眠状态。

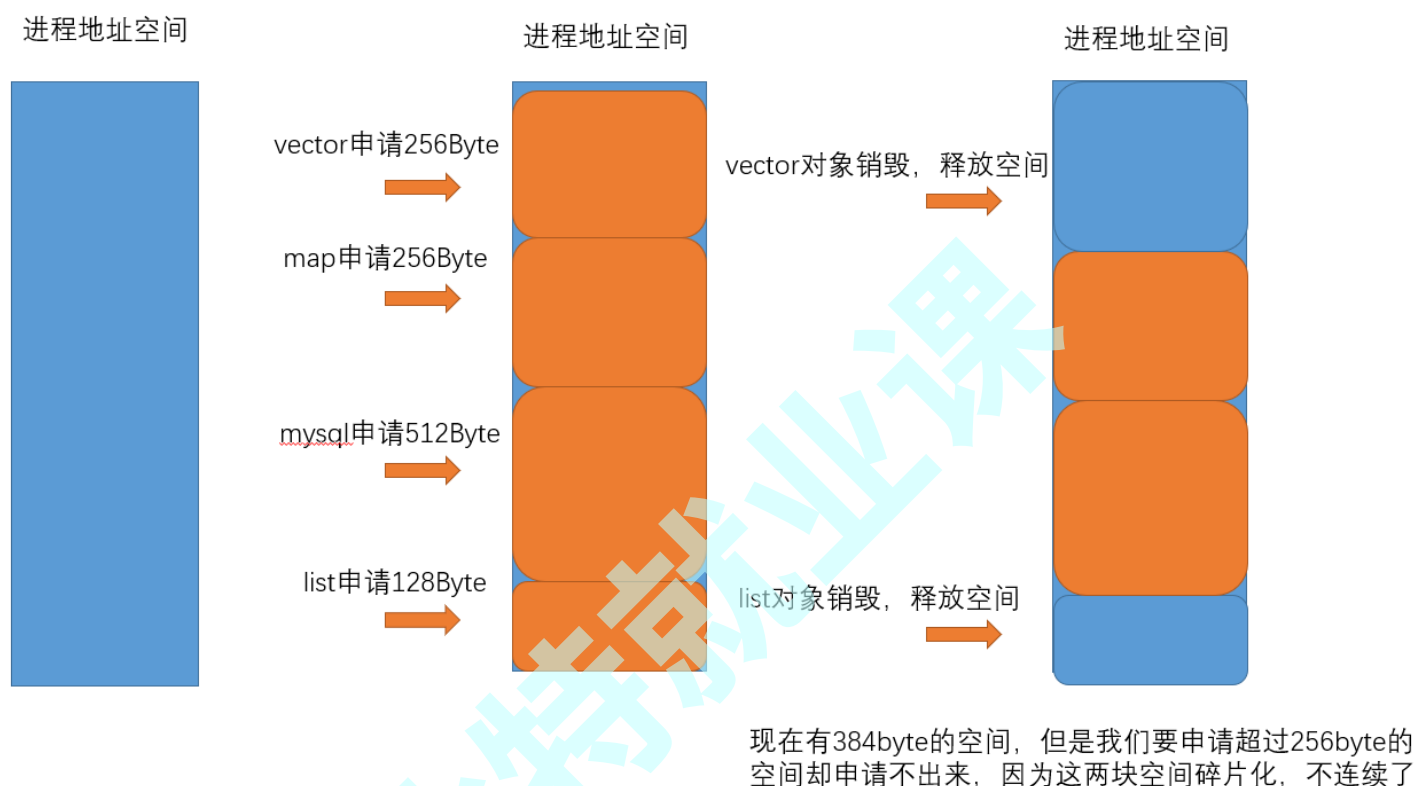
2.内存池

内存池是指程序预先从操作系统申请一块足够大内存，此后，当程序中需要申请内存的时候，不是直接向操作系统申请，而是直接从内存池中获取；同理，当程序释放内存的时候，并不真正将内存返回给操作系统，而是返回内存池。当程序退出(或者特定时间)时，内存池才将之前申请的内存真正释放。

3.内存池主要解决的问题

内存池主要解决的当然是效率的问题，其次如果作为系统的内存分配器的角度，还需要解决一下内存碎片的问题。那么什么是内存碎片呢？

再需要补充说明的是内存碎片分为外碎片和内碎片，上面我们讲的外碎片问题。外部碎片是一些空闲的连续内存区域太小，这些内存空间不连续，以至于合计的内存足够，但是不能满足一些的内存分配申请需求。内部碎片是由于一些对齐的需求，导致分配出去的空间中一些内存无法被利用。内碎片问题，我们后面项目就会看到，那会再进行更准确的理解。



4.malloc

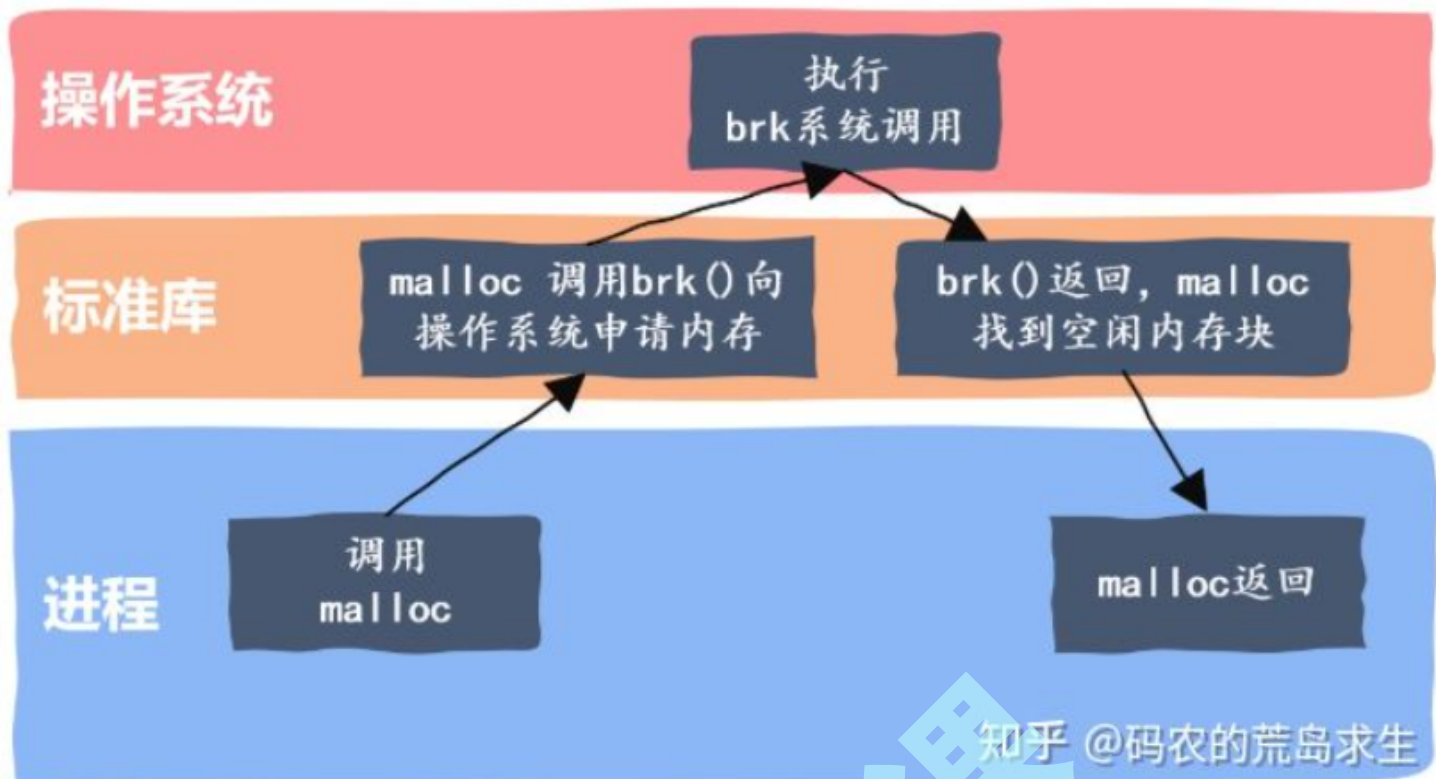
C/C++中我们要动态申请内存都是通过malloc去申请内存，但是我们要知道，实际我们不是直接去堆获取内存的，

而malloc就是一个内存池。malloc() 相当于向操作系统“批发”了一块较大的内存空间，然后“零售”给程序用。当全部“售完”或程序有大量的内存需求时，再根据实际需求向操作系统“进货”。malloc的实现方式有很多种，一般不同编译器平台用的都是不同的。比如windows的vs系列用的微软自己写的一套，linux gcc用的glibc中的ptmalloc。下面有几篇关于这块的文章，大概可以去简单看看了解一下，关于ptmalloc，学完我们的项目以后，有兴趣大家可以去看看他的实现细节。

[一文了解，Linux内存管理，malloc、free 实现原理](#)

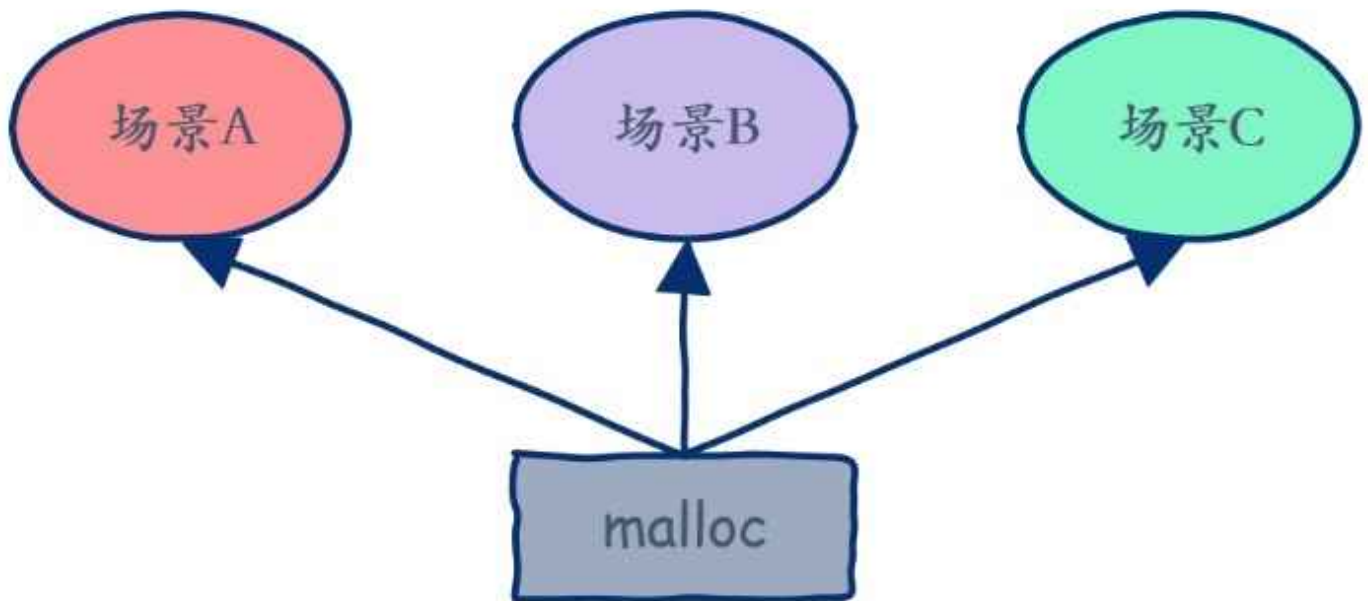
[malloc\(\)背后的实现原理——内存池](#)

[malloc的底层实现（ptmalloc）](#)

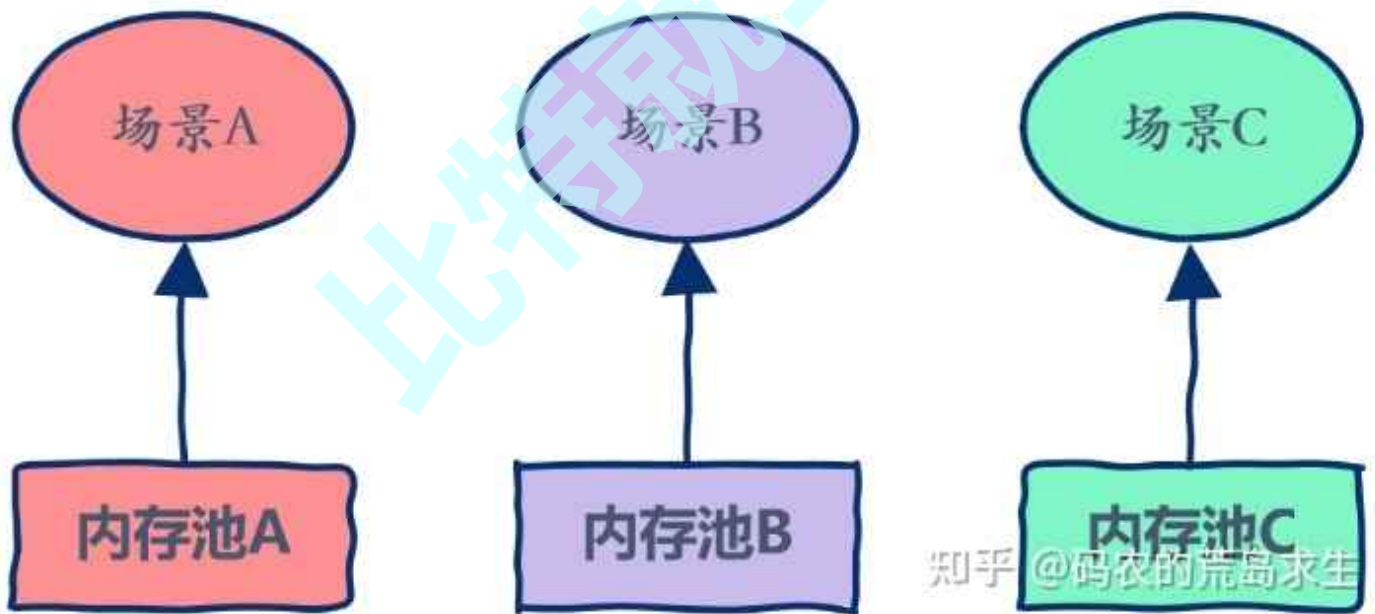


3.开胃菜--先设计一个定长的内存池

作为程序员(C/C++)我们知道申请内存使用的是malloc，malloc其实就是一个通用的大众货，什么场景下都可以用，但是什么场景下都可以用就意味着什么场景下都不会有很高的性能，下面我们就先来设计一个定长内存池做个开胃菜，当然这个定长内存池在我们后面的高并发内存池中也是有价值的，所以学习他目的有两层，先熟悉一下简单内存池是如何控制的，第二他会作为我们后面内存池的一个基础组件。



VS



windows和Linux下如何直接向堆申请页为单位的大块内存：

[VirtualAlloc](#)

[brk和mmap](#)

```
1 // 直接去堆上按页申请空间
2 inline static void* SystemAlloc(size_t kpage)
3 {
```

```

4  #ifdef _WIN32
5      void* ptr = VirtualAlloc(0, kpage*(1<<12), MEM_COMMIT | MEM_RESERVE,
        PAGE_READWRITE);
6  #else
7      // linux下brk mmap等
8  #endif
9
10     if (ptr == nullptr)
11         throw std::bad_alloc();
12
13     return ptr;
14 }
15
16 template<class T>
17 class ObjectPool
18 {
19 public:
20     T* New()
21     {
22         T* obj = nullptr;
23         // 如果自由链表有对象，直接取一个
24         if (_freeList)
25         {
26             obj = (T*)_freeList;
27             _freeList = *((void**)_freeList);
28         }
29         else
30         {
31             if (_leftBytes < sizeof(T))
32             {
33                 _leftBytes = 128 * 1024;
34                 //_memory = (char*)malloc(_leftBytes);
35                 _memory = (char*)SystemAlloc(_leftBytes);
36                 if (_memory == nullptr)
37                 {
38                     throw std::bad_alloc();
39                 }
40             }
41
42             obj = (T*)_memory;
43             size_t objSize = sizeof(T) < sizeof(void*) ? sizeof(void*) :
sizeof(T);
44             _memory += objSize;
45             _leftBytes -= objSize;
46         }
47
48         // 使用定位new调用T的构造函数初始化

```

```

49     new(obj)T;
50     return obj;
51 }
52
53 void Delete(T* obj)
54 {
55     // 显示调用的T的析构函数进行清理
56     obj->~T();
57
58     // 头插到freeList
59     *((void**)obj) = _freeList;
60
61     _freeList = obj;
62 }
63
64 private:
65     char* _memory = nullptr;    // 指向内存块的指针
66     int _leftBytes = 0;        // 内存块中剩余字节数
67     void* _freeList = nullptr; // 管理还回来的内存对象的自由链表
68 };
69
70 struct TreeNode
71 {
72     int _val;
73     TreeNode* _left;
74     TreeNode* _right;
75
76     TreeNode()
77         : _val(0)
78         , _left(nullptr)
79         , _right(nullptr)
80     {}
81 };
82
83 void TestObjectPool()
84 {
85     // 申请释放的轮次
86     const size_t Rounds = 3;
87
88     // 每轮申请释放多少次
89     const size_t N = 100000;
90
91     size_t begin1 = clock();
92     std::vector<TreeNode*> v1;
93     v1.reserve(N);
94
95     for (size_t j = 0; j < Rounds; ++j)

```



```

96     {
97         for (int i = 0; i < N; ++i)
98         {
99             v1.push_back(new TreeNode);
100         }
101         for (int i = 0; i < N; ++i)
102         {
103             delete v1[i];
104         }
105         v1.clear();
106     }
107
108     size_t end1 = clock();
109
110     ObjectPool<TreeNode> TNPool;
111     size_t begin2 = clock();
112     std::vector<TreeNode*> v2;
113     v2.reserve(N);
114
115     for (size_t j = 0; j < Rounds; ++j)
116     {
117         for (int i = 0; i < N; ++i)
118         {
119             v2.push_back(TNPool.New());
120         }
121         for (int i = 0; i < 100000; ++i)
122         {
123             TNPool.Delete(v2[i]);
124         }
125         v2.clear();
126     }
127     size_t end2 = clock();
128
129     cout << "new cost time:" << end1 - begin1 << endl;
130     cout << "object pool cost time:" << end2 - begin2 << endl;
131 }

```

4.高并发内存池整体框架设计

现代很多的开发环境都是多核多线程，在申请内存的场景下，必然存在激烈的锁竞争问题。malloc本身其实已经很优秀，那么我们项目的原型tcmalloc就是在多线程高并发的场景下更胜一筹，所以这次我们实现的内存池需要考虑以下几方面的问题。

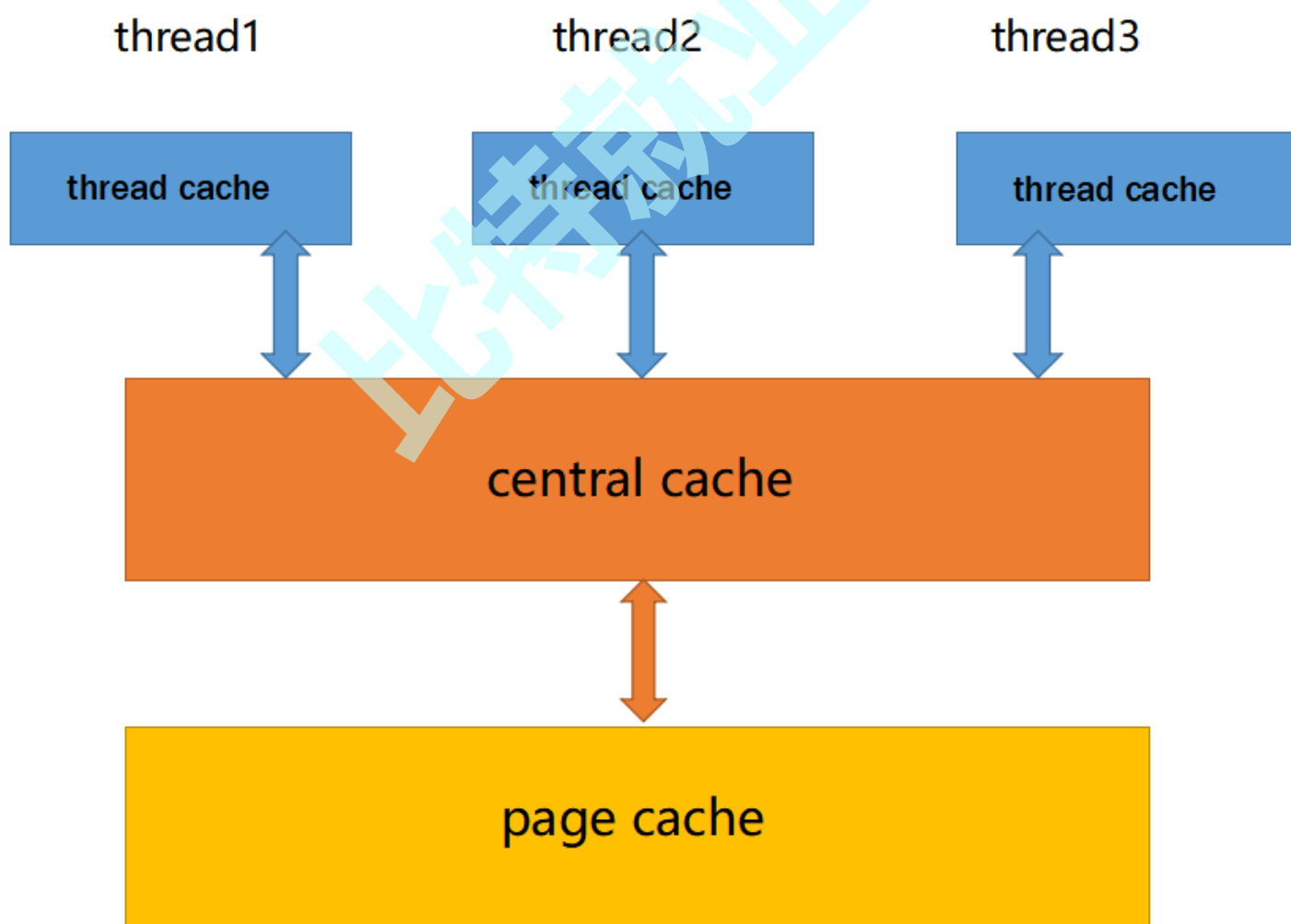
1. 性能问题。

2. 多线程环境下，锁竞争问题。

3. 内存碎片问题。

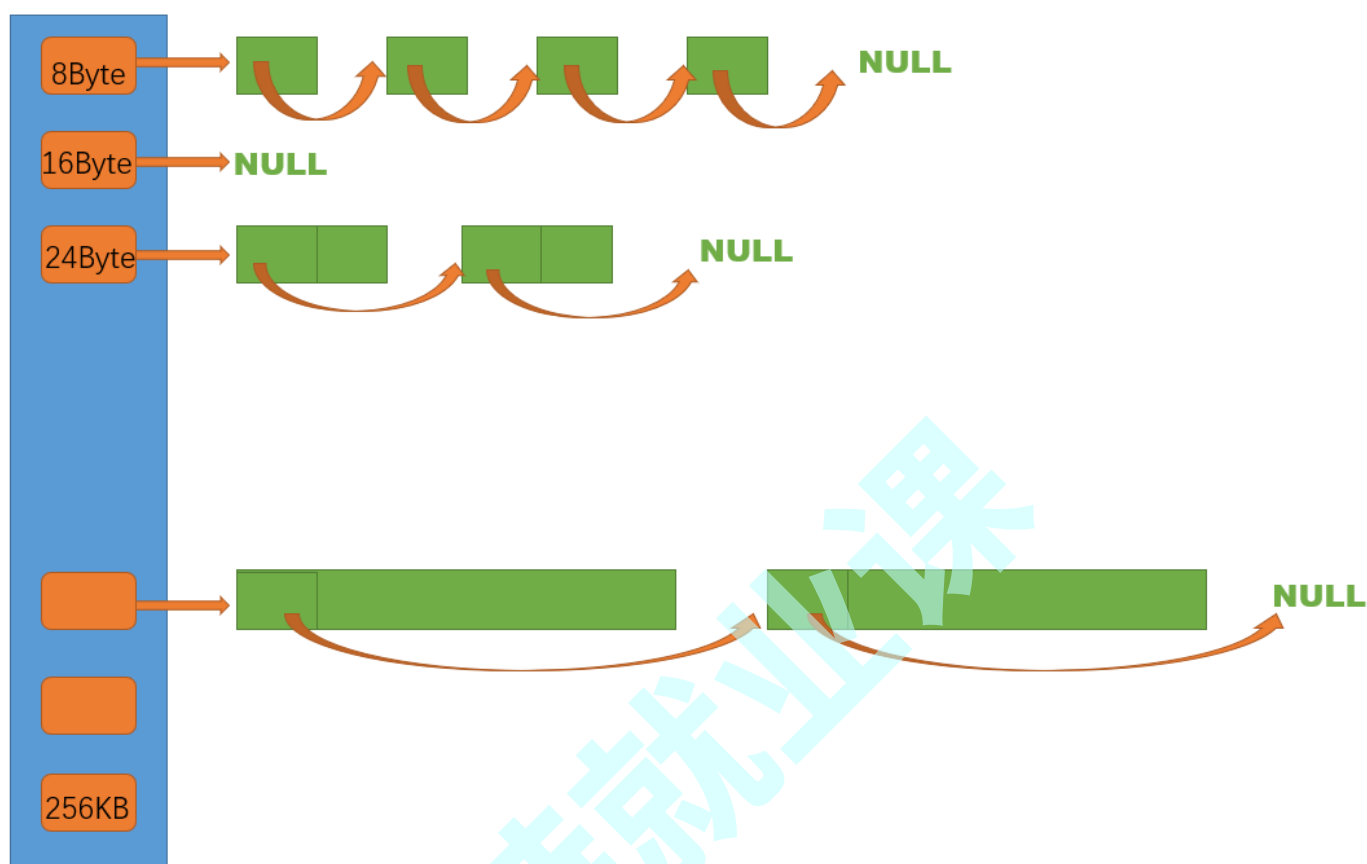
concurrent memory pool主要由以下3个部分构成：

1. **thread cache**：线程缓存是每个线程独有的，用于小于256KB的内存的分配，**线程从这里申请内存不需要加锁，每个线程独享一个cache，这也就是这个并发线程池高效的地方。**
2. **central cache**：中心缓存是所有线程所共享，thread cache是**按需从central cache中获取**的对象。central cache合适的时机回收thread cache中的对象，避免一个线程占用了太多的内存，而其他线程的内存吃紧，**达到内存分配在多个线程中更均衡的按需调度的目的。**central cache是存在竞争的，所以从这里取内存对象是需要加锁，**首先这里用的是桶锁，其次只有thread cache的没有内存对象时才会找central cache，所以这里竞争不会很激烈。**
3. **page cache**：页缓存是在central cache缓存上面的一层缓存，存储的内存是以页为单位存储及分配的，central cache没有内存对象时，从page cache分配出一定数量的page，并切割成定长大小的小块内存，分配给central cache。**当一个span的几个跨度页的对象都回收以后，page cache会回收central cache满足条件的span对象，并且合并相邻的页，组成更大的页，缓解内存碎片的问题。**



5.高并发内存池--thread cache

thread cache是哈希桶结构，每个桶是一个按桶位置映射大小的内存块对象的自由链表。每个线程都会有一个thread cache对象，这样每个线程在这里获取对象和释放对象时是无锁的。



申请内存：

1. 当内存申请size<=256KB，先获取到线程本地存储的thread cache对象，计算size映射的哈希桶自由链表下标i。
2. 如果自由链表_freeLists[i]中有对象，则直接Pop一个内存对象返回。
3. 如果_freeLists[i]中没有对象时，则批量从central cache中获取一定数量的对象，插入到自由链表并返回一个对象。

释放内存：

1. 当释放内存小于256k时将内存释放回thread cache，计算size映射自由链表桶位置i，将对象Push到_freeLists[i]。
2. 当链表的长度过长，则回收一部分内存对象到central cache。

TLS--thread local storage:

linux gcc下tls

thread cache代码框架:

```
1 // thread cache本质是由一个哈希映射的对象自由链表构成
2 class ThreadCache
3 {
4 public:
5     // 申请和释放内存对象
6     void* Allocate(size_t size);
7     void Deallocate(void* ptr, size_t size);
8
9     // 从中心缓存获取对象
10    void* FetchFromCentralCache(size_t index, size_t size);
11
12    // 释放对象时, 链表过长时, 回收内存回到中心缓存
13    void ListTooLong(FreeList& list, size_t size);
14 private:
15     FreeList _freeLists[NFREELIST];
16 };
17
18 // TLS thread local storage
19 static _declspec(thread) ThreadCache* pTLSThreadCache = nullptr;
20
21 // 管理小对象的自由链表
22 // 管理切分好的小对象的自由链表
23 class FreeList
24 {
25 public:
26     void Push(void* obj)
27     {
28         assert(obj);
29
30         // 头插
31         /*(void**)obj = _freeList;
32         NextObj(obj) = _freeList;
33         _freeList = obj;
34
35         ++_size;
36     }
37
38     void PushRange(void* start, void* end, size_t n)
39     {
40         NextObj(end) = _freeList;
41         _freeList = start;
42
43         _size += n;
44     }
```

```
45
46 void PopRange(void*& start, void*& end, size_t n)
47 {
48     assert(n >= _size);
49     start = _freeList;
50     end = start;
51
52     for (size_t i = 0; i < n - 1; ++i)
53     {
54         end = NextObj(end);
55     }
56
57     _freeList = NextObj(end);
58     NextObj(end) = nullptr;
59     _size -= n;
60 }
61
62 void* Pop()
63 {
64     assert(_freeList);
65
66     // 头删
67     void* obj = _freeList;
68     _freeList = NextObj(obj);
69     --_size;
70
71     return obj;
72 }
73
74 bool Empty()
75 {
76     return _freeList == nullptr;
77 }
78
79 size_t& MaxSize()
80 {
81     return _maxSize;
82 }
83
84 size_t Size()
85 {
86     return _size;
87 }
88
89 private:
90 void* _freeList = nullptr;
91 size_t _maxSize = 1;
```

```
92     size_t _size = 0;
93 };
```

自由链表的哈希桶跟对象大小的映射关系

```
1 // 小于等于MAX_BYTES, 就找thread cache申请
2 // 大于MAX_BYTES, 就直接找page cache或者系统堆申请
3 static const size_t MAX_BYTES = 256 * 1024;
4 // thread cache 和 central cache自由链表哈希桶的表大小
5 static const size_t NFREELISTS = 208;
6 // page cache 管理span list哈希表大小
7 static const size_t NPAGES = 129;
8 // 页大小转换偏移, 即一页定义为2^13, 也就是8KB
9 static const size_t PAGE_SHIFT = 13;
10
11 // 地址大小类型, 32位下是4byte类型, 64位下是8byte类型
12 #ifdef _WIN32
13     typedef size_t ADDRES_INT;
14 #else
15     typedef unsigned long long ADDRES_INT;
16 #endif // _WIN32
17
18 // 页编号类型, 32位下是4byte类型, 64位下是8byte类型
19 #ifdef _WIN32
20     typedef size_t PageID;
21 #else
22     typedef unsigned long long PageID;
23 #endif // _WIN32
24
25 // 获取内存对象中存储的头4 or 8字节值, 即链接的下一个对象的地址
26 inline void*& NextObj(void* obj)
27 {
28     return *((void**)obj);
29 }
30
31 // 管理对齐和映射等关系
32 class SizeClass
33 {
34 public:
35     // 整体控制在最多10%左右的内碎片浪费
36     // [1,128]                8byte对齐        freelist[0,16)
37     // [128+1,1024]          16byte对齐        freelist[16,72)
38     // [1024+1,81024]        128byte对齐       freelist[72,128)
```

```

39 * // [8*1024+1,64*1024]      1024byte对齐      freelist[128,184)
40 * // [64*1024+1,256*1024]    8*1024byte对齐    freelist[184,208)
41 static inline size_t _RoundUp(size_t bytes, size_t align)
42 {
43     return (((bytes)+align - 1) & ~(align - 1));
44 }
45
46 // 对齐大小计算
47 static inline size_t RoundUp(size_t bytes)
48 {
49     if (bytes <= 128){
50         return _RoundUp(bytes, 8);
51     }
52     else if (bytes <= 1024){
53         return _RoundUp(bytes, 16);
54     }
55     else if (bytes <= 81024){
56         return _RoundUp(bytes, 128);
57     }
58 *     else if (bytes <= 64*1024){
59         return _RoundUp(bytes, 1024);
60     }
61     else if (bytes <= 2561024){
62 *         return _RoundUp(bytes, 8*1024);
63     }
64     else{
65         return _RoundUp(bytes, 1 << PAGE_SHIFT);
66     }
67
68     return -1;
69 }
70
71 static inline size_t _Index(size_t bytes, size_t align_shift)
72 {
73     return ((bytes + (1 << align_shift) - 1) >> align_shift) - 1;
74 }
75
76 // 计算映射的哪一个自由链表桶
77 static inline size_t Index(size_t bytes)
78 {
79     assert(bytes <= MAX_BYTES);
80
81     // 每个区间有多少个链
82     static int group_array[4] = { 16, 56, 56, 56 };
83     if (bytes <= 128){
84         return _Index(bytes, 3);
85     }

```

```

86     else if (bytes <= 1024){
87         return _Index(bytes - 128, 4) + group_array[0];
88     }
89     else if (bytes <= 81024){
90         return _Index(bytes - 1024, 7) + group_array[1] + group_array[0];
91     }
92 *     else if (bytes <= 64*1024){
93         return _Index(bytes - 8*1024, 10) + group_array[2] +
group_array[1] + group_array[0];
94     }
95     else if (bytes <= 256 * 1024){
96         return _Index(bytes - 64 * 1024, 13) + group_array[3] +
group_array[2] + group_array[1] + group_array[0];
97     }
98     else{
99         assert(false);
100    }
101
102    return -1;
103 }
104
105 // 一次从中心缓存获取多少个
106 static size_t NumMoveSize(size_t size)
107 {
108     if (size == 0)
109         return 0;
110
111     // [2, 512], 一次批量移动多少个对象的(慢启动)上限值
112     // 小对象一次批量上限高
113     // 小对象一次批量上限低
114     int num = MAX_BYTES / size;
115     if (num < 2)
116         num = 2;
117
118     if (num > 512)
119         num = 512;
120
121     return num;
122 }
123
124 // 计算一次向系统获取几个页
125 // 单个对象 8byte
126 // ...
127 // 单个对象 256KB
128 static size_t NumMovePage(size_t size)
129 {
130     size_t num = NumMoveSize(size);

```



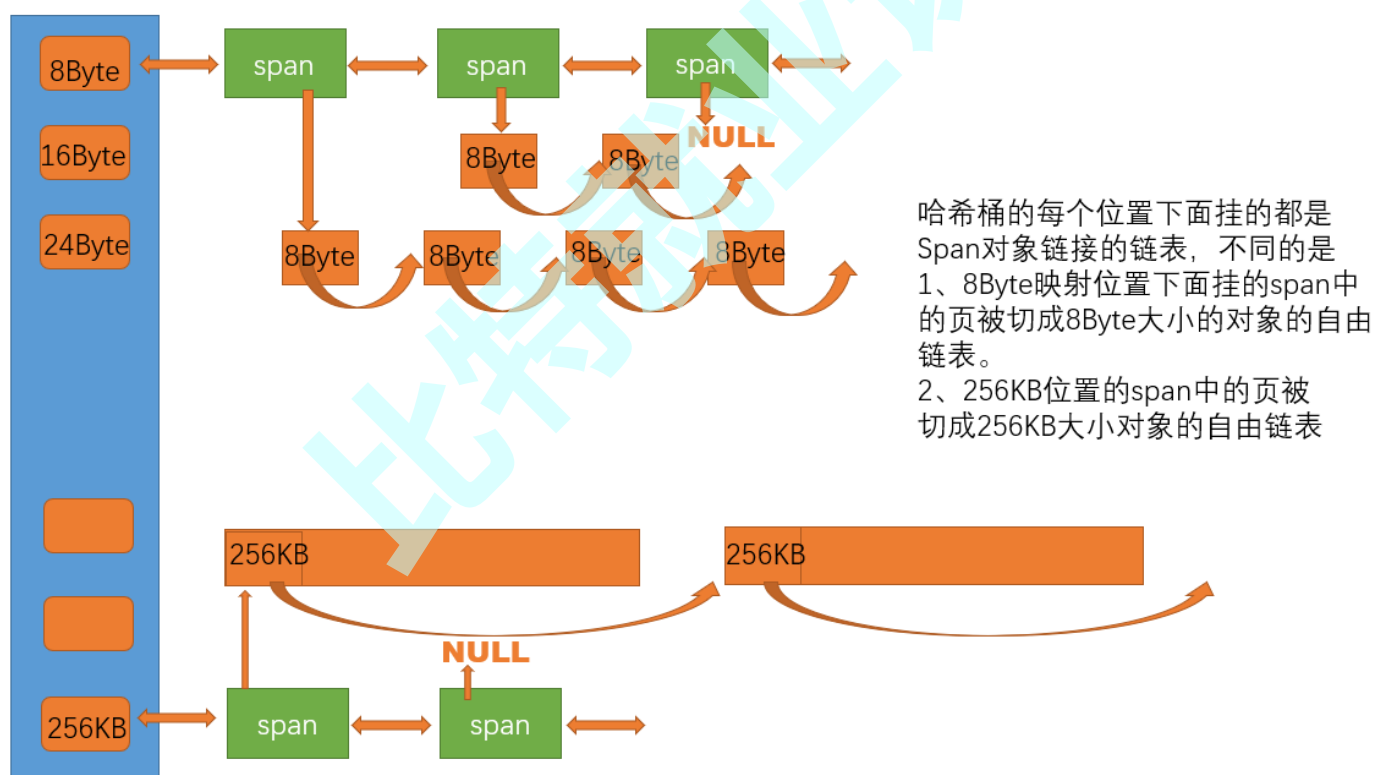
```

131     size_t npage = num*size;
132
133     npage >>= PAGE_SHIFT;
134     if (npage == 0)
135         npage = 1;
136
137     return npage;
138 }
139 };

```

5.高并发内存池--central cache

central cache也是一个哈希桶结构，他的哈希桶的映射关系跟thread cache是一样的。不同的是他的每个哈希桶位置挂是SpanList链表结构，不过每个映射桶下面的span中的大内存块被按映射关系切成了一个个小内存块对象挂在span的自由链表中。



申请内存：

1. 当thread cache中没有内存时，就会批量向central cache申请一些内存对象，这里的批量获取对象的数量使用了类似网络tcp协议拥塞控制的慢开始算法；central cache也有一个哈希映射的spanlist，spanlist中挂着span，从span中取出对象给thread cache，这个过程是需要加锁的，不过这里使用的是一个桶锁，尽可能提高效率。

2. central cache映射的spanlist中所有span的都没有内存以后，则需要向page cache申请一个新的span对象，拿到span以后将span管理的内存按大小切好作为自由链表链接到一起。然后从span中取对象给thread cache。
3. central cache的中挂的span中use_count记录分配了多少个对象出去，分配一个对象给thread cache，就++use_count

释放内存：

1. 当thread_cache过长或者线程销毁，则会将内存释放回central cache中的，释放回来时--use_count。当use_count减到0时则表示所有对象都回到了span，则将span释放回page cache，page cache中会对前后相邻的空闲页进行合并。

CentralCache 代码框架：

```
1 // 单例模式
2 class CentralCache
3 {
4 public:
5     static CentralCache* GetInstance()
6     {
7         return &_sInst;
8     }
9
10    // 获取一个非空的span
11    Span* GetOneSpan(SpanList& list, size_t byte_size);
12
13    // 从中心缓存获取一定数量的对象给thread cache
14    size_t FetchRangeObj(void*& start, void*& end, size_t batchNum, size_t
size);
15
16    // 将一定数量的对象释放到span跨度
17    void ReleaseListToSpans(void* start, size_t byte_size);
18 private:
19     SpanList _spanLists[NFREELIST];
20
21 private:
22     CentralCache()
23     {}
24
25     CentralCache(const CentralCache&) = delete;
26
27     static CentralCache _sInst;
28 };
```

以页为单位的大内存管理span的定义及spanlist定义

```
1 // Span管理一个跨度的大块内存
2
3 // 管理以页为单位的大块内存
4 // 管理多个连续页大块内存跨度结构
5 struct Span
6 {
7     PAGE_ID _pageId = 0; // 大块内存起始页的页号
8     size_t _n = 0;       // 页的数量
9
10    Span* _next = nullptr; // 双向链表的结构
11    Span* _prev = nullptr;
12
13    size_t _objSize = 0; // 切好的小对象的大小
14    size_t _useCount = 0; // 切好小块内存，被分配给thread cache的计数
15    void* _freeList = nullptr; // 切好的小块内存的自由链表
16
17    bool _isUse = false; // 是否在被使用
18 };
19
20 // 带头双向循环链表
21 class SpanList
22 {
23 public:
24     SpanList()
25     {
26         _head = new Span;
27         _head->_next = _head;
28         _head->_prev = _head;
29     }
30
31     Span* Begin()
32     {
33         return _head->_next;
34     }
35
36     Span* End()
37     {
38         return _head;
39     }
40
41     bool Empty()
42     {
```

```

43         return _head->_next == _head;
44     }
45
46     void PushFront(Span* span)
47     {
48         Insert(Begin(), span);
49     }
50
51     Span* PopFront()
52     {
53         Span* front = _head->_next;
54         Erase(front);
55         return front;
56     }
57
58     void Insert(Span* pos, Span* newSpan)
59     {
60         assert(pos);
61         assert(newSpan);
62
63         Span* prev = pos->_prev;
64         // prev newspan pos
65         prev->_next = newSpan;
66         newSpan->_prev = prev;
67         newSpan->_next = pos;
68         pos->_prev = newSpan;
69     }
70
71     void Erase(Span* pos)
72     {
73         assert(pos);
74         assert(pos != _head);
75
76         Span* prev = pos->_prev;
77         Span* next = pos->_next;
78
79         prev->_next = next;
80         next->_prev = prev;
81     }
82
83 private:
84     Span* _head;
85 public:
86     std::mutex _mtx; // 桶锁
87 };

```

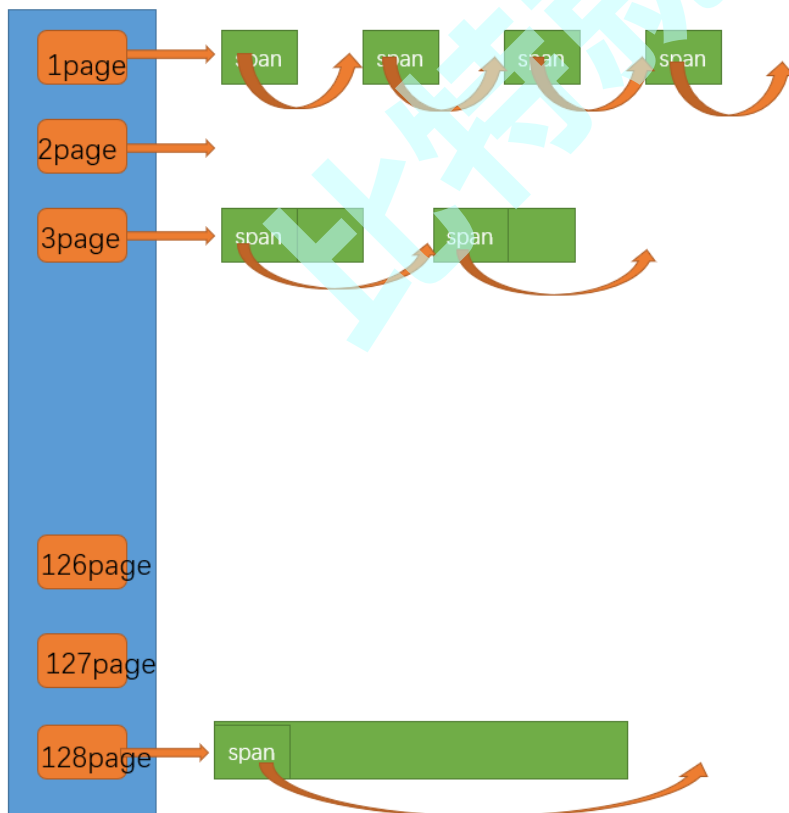
5.高并发内存池--page cache

申请内存：

1. 当central cache向page cache申请内存时，**page cache先检查对应位置有没有span，如果没有则向更大页寻找一个span，如果找到则分裂成两个**。比如：申请的是4页page，4页page后面没有挂span，则向后面寻找更大的span，假设在10页page位置找到一个span，则将10页page span分裂为一个4页page span和一个6页page span。
2. 如果找到_spanList[128]都没有合适的span，则向系统使用mmap、brk或者是VirtualAlloc等方式申请128页page span挂在自由链表中，再重复1中的过程。
3. 需要注意的是central cache和page cache 的核心结构都是spanlist的哈希桶，但是他们是有本质区别的，central cache中哈希桶，是按跟thread cache一样的大小对齐关系映射的，他的spanlist中挂的span中的内存都被按映射关系切好链接成小块内存的自由链表。而page cache 中的spanlist则是按下标桶号映射的，也就是说第i号桶中挂的span都是i页内存。

释放内存：

1. 如果central cache释放回一个span，则依次寻找span的前后page id的没有在使用的空闲span，看是否可以合并，如果合并继续向前寻找。这样就可以将切小的内存合并收缩成大的span，减少内存碎片。



PageCache 代码框架：

```

1 // 1.page cache是一个以页为单位的span自由链表
2 // 2.为了保证全局只有唯一的page cache，这个类被设计成了单例模式。
3 class PageCache
4 {
5 public:
6     static PageCache* GetInstance()
7     {
8         return &_sInst;
9     }
10
11     // 获取从对象到span的映射
12     Span* MapObjectToSpan(void* obj);
13
14     // 释放空闲span回到Pagecache，并合并相邻的span
15     void ReleaseSpanToPageCache(Span* span);
16
17     // 获取一个K页的span
18     Span* NewSpan(size_t k);
19
20     std::mutex _pageMtx;
21 private:
22     SpanList _spanLists[NPAGES];
23     ObjectPool<Span> _spanPool;
24
25     //std::unordered_map<PAGE_ID, Span*> _idSpanMap;
26     //std::map<PAGE_ID, Span*> _idSpanMap;
27     TCMalloc_PageMap1<32 - PAGE_SHIFT> _idSpanMap;
28
29     PageCache()
30     {}
31     PageCache(const PageCache&) = delete;
32
33
34     static PageCache _sInst;
35 };

```

windows和Linux下如何直接向堆申请页为单位的大块内存：

VirtualAlloc

brk和mmap

```

1 inline static void* SystemAlloc(size_t kpage)
2 {

```

```

3  #ifdef _WIN32
4      void* ptr = VirtualAlloc(0, kpage*(1 << PAGE_SHIFT),
5          MEM_COMMIT | MEM_RESERVE, PAGE_READWRITE);
6  #else
7      // brk mmap等
8  #endif
9
10     if (ptr == nullptr)
11         throw std::bad_alloc();
12
13     return ptr;
14 }
15
16 inline static void SystemFree(void* ptr)
17 {
18     #ifdef _WIN32
19         VirtualFree(ptr, 0, MEM_RELEASE);
20     #else
21         // sbrk unmmap等
22     #endif
23 }

```

6.多线程并发环境下，对比malloc和ConcurrentAlloc申请和释放内存效率对比

```

1  void BenchmarkMalloc(size_t ntimes, size_t nworks, size_t rounds)
2  {
3      std::vector<std::thread> vthread(nworks);
4      std::atomic<size_t> malloc_costtime = 0;
5      std::atomic<size_t> free_costtime = 0;
6
7      for (size_t k = 0; k < nworks; ++k)
8      {
9          vthread[k] = std::thread(&, k {
10              std::vector<void*> v;
11              v.reserve(ntimes);
12
13              for (size_t j = 0; j < rounds; ++j)
14              {
15                  size_t begin1 = clock();
16                  for (size_t i = 0; i < ntimes; i++)
17                  {
18                      v.push_back(malloc(16));
19                      //v.push_back(malloc((16 + i) % 8192 + 1));

```

```

20         }
21         size_t end1 = clock();
22
23         size_t begin2 = clock();
24         for (size_t i = 0; i < ntimes; i++)
25         {
26             free(v[i]);
27         }
28         size_t end2 = clock();
29         v.clear();
30
31         malloc_costtime += (end1 - begin1);
32         free_costtime += (end2 - begin2);
33     }
34 });
35 }
36
37 for (auto& t : vthread)
38 {
39     t.join();
40 }
41
42 printf("%u个线程并发执行%u轮次, 每轮次malloc %u次: 花费: %u ms\n",
43        nworks, rounds, ntimes, malloc_costtime);
44
45 printf("%u个线程并发执行%u轮次, 每轮次free %u次: 花费: %u ms\n",
46        nworks, rounds, ntimes, free_costtime);
47
48 printf("%u个线程并发malloc&free %u次, 总计花费: %u ms\n",
49        nworks, nworks*rounds*ntimes, malloc_costtime + free_costtime);
50 }
51
52
53 // 单轮次申请释放次数 线程数 轮次
54 void BenchmarkConcurrentMalloc(size_t ntimes, size_t nworks, size_t rounds)
55 {
56     std::vector<std::thread>(std::thread) vthread(nworks);
57     std::atomic<size_t> malloc_costtime = 0;
58     std::atomic<size_t> free_costtime = 0;
59
60     for (size_t k = 0; k < nworks; ++k)
61     {
62         vthread[k] = std::thread(& {
63             std::vector<void*> v;
64             v.reserve(ntimes);
65
66             for (size_t j = 0; j < rounds; ++j)

```



```

67         {
68             size_t begin1 = clock();
69             for (size_t i = 0; i < ntimes; i++)
70             {
71                 v.push_back(ConcurrentAlloc(16));
72                 //v.push_back(ConcurrentAlloc((16 + i) % 8192 + 1));
73             }
74             size_t end1 = clock();
75
76             size_t begin2 = clock();
77             for (size_t i = 0; i < ntimes; i++)
78             {
79                 ConcurrentFree(v[i]);
80             }
81             size_t end2 = clock();
82             v.clear();
83
84             malloc_costtime += (end1 - begin1);
85             free_costtime += (end2 - begin2);
86         }
87     });
88 }
89
90 for (auto& t : vthread)
91 {
92     t.join();
93 }
94
95 printf("%u个线程并发执行%u轮次, 每轮次concurrent alloc %u次: 花费: %u ms\n",
96        nworks, rounds, ntimes, malloc_costtime);
97
98 printf("%u个线程并发执行%u轮次, 每轮次concurrent dealloc %u次: 花费: %u ms\n",
99        nworks, rounds, ntimes, free_costtime);
100
101 printf("%u个线程并发concurrent alloc&dealloc %u次, 总计花费: %u ms\n",
102        nworks, nworks*rounds*ntimes, malloc_costtime + free_costtime);
103 }
104
105 int main()
106 {
107     size_t n = 10000;
108     cout << "===== " <<
endl;
109     BenchmarkConcurrentMalloc(n, 4, 10);
110     cout << endl << endl;
111
112     BenchmarkMalloc(n, 4, 10);

```

```

113     cout << "===== " <<
endl;
114
115     return 0;
116 }

```

7.使用tcmalloc源码中实现基数树进行优化

```

1 // Single-level array
2 template <int BITS>
3 class TCMalloc_PageMap1 {
4 private:
5     static const int LENGTH = 1 << BITS;
6     void** array_;
7
8 public:
9     typedef uintptr_t Number;
10
11     explicit TCMalloc_PageMap1(void* (allocator)(size_t)) {
12 *         array_ = reinterpret_cast<void**>((*allocator)(sizeof(void*)
<< BITS));
13         memset(array_, 0, sizeof(void*) << BITS);
14     }
15
16     // Return the current value for KEY. Returns NULL if not yet set,
17     // or if k is out of range.
18     void* get(Number k) const {
19         if ((k >> BITS) > 0) {
20             return NULL;
21         }
22         return array_[k];
23     }
24
25     // REQUIRES "k" is in range "[0, 2^BITS-1]".
26     // REQUIRES "k" has been ensured before.
27     //
28     // Sets the value 'v' for key 'k'.
29     void set(Number k, void* v) {
30         array_[k] = v;
31     }
32 };
33
34 // Two-level radix tree

```

```

35 template <int BITS>
36 class TCMalloc_PageMap2 {
37 private:
38     // Put 32 entries in the root and (2^BITS)/32 entries in each leaf.
39     static const int ROOT_BITS = 5;
40     static const int ROOT_LENGTH = 1 << ROOT_BITS;
41
42     static const int LEAF_BITS = BITS - ROOT_BITS;
43     static const int LEAF_LENGTH = 1 << LEAF_BITS;
44
45     // Leaf node
46     struct Leaf {
47         void* values[LEAF_LENGTH];
48     };
49
50     Leaf* root_[ROOT_LENGTH];           // Pointers to 32 child nodes
51     void* (*allocator_)(size_t);       // Memory allocator
52
53 public:
54     typedef uintptr_t Number;
55
56     explicit TCMalloc_PageMap2(void* (*allocator)(size_t)) {
57         allocator_ = allocator;
58         memset(root_, 0, sizeof(root_));
59     }
60
61     void* get(Number k) const {
62         const Number i1 = k >> LEAF_BITS;
63         const Number i2 = k & (LEAF_LENGTH - 1);
64         if ((k >> BITS) > 0 || root_[i1] == NULL) {
65             return NULL;
66         }
67         return root_[i1]->values[i2];
68     }
69
70     void set(Number k, void* v) {
71         const Number i1 = k >> LEAF_BITS;
72         const Number i2 = k & (LEAF_LENGTH - 1);
73         ASSERT(i1 < ROOT_LENGTH);
74         root_[i1]->values[i2] = v;
75     }
76
77     bool Ensure(Number start, size_t n) {
78         for (Number key = start; key <= start + n - 1;) {
79             const Number i1 = key >> LEAF_BITS;
80
81             // Check for overflow

```

```

82         if (i1 >= ROOT_LENGTH)
83             return false;
84
85         // Make 2nd level node if necessary
86         if (root_[i1] == NULL) {
87             Leaf* leaf = reinterpret_cast<Leaf*>
88             ((*allocator_)(sizeof(Leaf)));
89             if (leaf == NULL) return false;
90             memset(leaf, 0, sizeof(*leaf));
91             root_[i1] = leaf;
92         }
93
94         // Advance key past whatever is covered by this leaf
95         node
96         key = ((key >> LEAF_BITS) + 1) << LEAF_BITS;
97     }
98     return true;
99 }
100
101 void PreallocateMoreMemory() {
102     // Allocate enough to keep track of all possible pages
103     Ensure(0, 1 << BITS);
104 }
105 };
106
107 // Three-level radix tree
108 template <int BITS>
109 class TCMalloc_PageMap3 {
110 private:
111     // How many bits should we consume at each interior level
112     static const int INTERIOR_BITS = (BITS + 2) / 3; // Round-up
113     static const int INTERIOR_LENGTH = 1 << INTERIOR_BITS;
114
115     // How many bits should we consume at leaf level
116     static const int LEAF_BITS = BITS - 2 * INTERIOR_BITS;
117     static const int LEAF_LENGTH = 1 << LEAF_BITS;
118
119     // Interior node
120     struct Node {
121         Node* ptrs[INTERIOR_LENGTH];
122     };
123
124     // Leaf node
125     struct Leaf {
126         void* values[LEAF_LENGTH];
127     };

```

```

127     Node* root_; // Root of radix tree
128     void* (*allocator_)(size_t); // Memory allocator
129
130     Node* NewNode() {
131         Node* result = reinterpret_cast<Node*>((*allocator_)(
132             sizeof(Node)));
133         if (result != NULL) {
134             memset(result, 0, sizeof(*result));
135         }
136         return result;
137     }
138 public:
139     typedef uintptr_t Number;
140
141     explicit TCMalloc_PageMap3(void* (*allocator)(size_t)) {
142         allocator_ = allocator;
143         root_ = NewNode();
144     }
145
146     void* get(Number k) const {
147         const Number i1 = k >> (LEAF_BITS + INTERIOR_BITS);
148         const Number i2 = (k >> LEAF_BITS) & (INTERIOR_LENGTH - 1);
149         const Number i3 = k & (LEAF_LENGTH - 1);
150         if ((k >> BITS) > 0 ||
151             root_>ptrs[i1] == NULL || root_>ptrs[i1]>ptrs[i2]
152 == NULL) {
153             return NULL;
154         }
155         return reinterpret_cast<Leaf*>(root_>ptrs[i1]>ptrs[i2])>
156         values[i3];
157     }
158
159     void set(Number k, void* v) {
160         ASSERT(k >> BITS == 0);
161         const Number i1 = k >> (LEAF_BITS + INTERIOR_BITS);
162         const Number i2 = (k >> LEAF_BITS) & (INTERIOR_LENGTH - 1);
163         const Number i3 = k & (LEAF_LENGTH - 1);
164         reinterpret_cast<Leaf*>(root_>ptrs[i1]>ptrs[i2])>values[i3]
165 = v;
166     }
167
168     bool Ensure(Number start, size_t n) {
169         for (Number key = start; key <= start + n - 1; key++) {
170             const Number i1 = key >> (LEAF_BITS + INTERIOR_BITS);
171             const Number i2 = (key >> LEAF_BITS) &
172 (INTERIOR_LENGTH - 1);

```

```

169
170         // Check for overflow
171         if (i1 >= INTERIOR_LENGTH || i2 >= INTERIOR_LENGTH)
172             return false;
173
174         // Make 2nd level node if necessary
175         if (root_>ptrs[i1] == NULL) {
176             Node* n = NewNode();
177             if (n == NULL) return false;
178             root_>ptrs[i1] = n;
179         }
180
181         // Make leaf node if necessary
182         if (root_>ptrs[i1]>ptrs[i2] == NULL) {
183             Leaf* leaf = reinterpret_cast<Leaf*>
184             ((*allocator_)(sizeof(Leaf)));
185             if (leaf == NULL) return false;
186             memset(leaf, 0, sizeof(leaf));
187             *
188             root_>ptrs[i1]>ptrs[i2] =
189             reinterpret_cast<Node*>(leaf);
190         }
191         // Advance key past whatever is covered by this leaf
192         key = ((key >> LEAF_BITS) + 1) << LEAF_BITS;
193     }
194     return true;
195 }
196
197 void PreallocateMoreMemory() {
198 }
199 };

```

8. 扩展学习及当前项目实现的不足

实际中我们测试了，当前实现的并发内存池比malloc/free是更加高效的，那么我们能否替换到系统调用malloc呢？实际上是可以的。

- 不同平台替换方式不同。基于unix的系统上的glibc，使用了weak alias的方式替换。具体来说是因为这些入口函数都被定义成了weak symbols，再加上gcc支持 alias attribute，所以替换就变成了这种通用形式：

```
void* malloc(size_t size) THROW attribute__((alias (tc_malloc)))
```

因此所有malloc的调用都跳转到了tc_malloc的实现

具体参考这里：[GCC attribute 之weak,alias属性](#)

有些平台不支持这样的东西，需要使用hook的钩子技术来做。

关于hook请看这里：[hook](#)

调研参考资料

[几个内存池库的对比](#)

[tcmalloc源码学习](#)

[TCMALLOC 源码阅读](#)

[如何设计内存池？ - 码农的荒岛求生的回答 - 知乎](#)

[tcmalloc源代码](#)

比特就业课